

Universidad Autónoma Metropolitana

División de Ciencias Básicas e Ingeniería

# Seminario de Tecnologías II

## Tarea 3

Gráficos de Bajo Nivel: WebGL2, WebGPU y TSL

Diego Barrera Hernández

Brandon Daniel Martínez Alcántara

Ciudad de México, 2025

# Índice

|                                                                        |          |
|------------------------------------------------------------------------|----------|
| <b>1. Objetivo</b>                                                     | <b>3</b> |
| <b>2. Escenas implementadas</b>                                        | <b>3</b> |
| 2.1. Brazo robótico articulado . . . . .                               | 3        |
| 2.2. Sistema solar . . . . .                                           | 3        |
| <b>3. Biblioteca compartida</b>                                        | <b>4</b> |
| 3.1. <code>mat4.js</code> — Álgebra de matrices $4 \times 4$ . . . . . | 4        |
| 3.2. <code>MatrixStack.js</code> — Pila de matrices . . . . .          | 4        |
| 3.3. <code>primitives.js</code> — Generadores de geometría . . . . .   | 4        |
| 3.4. <code>controls-ui.js</code> — Panel de controles . . . . .        | 5        |
| <b>4. Parte A — WebGL2 + GLSL</b>                                      | <b>5</b> |
| 4.1. Pipeline manual . . . . .                                         | 5        |
| 4.2. VAOs y VBOs . . . . .                                             | 5        |
| 4.3. Shader GLSL ES 3.00 . . . . .                                     | 5        |
| <b>5. Parte B — WebGPU + WGSL</b>                                      | <b>6</b> |
| 5.1. Inicialización del dispositivo . . . . .                          | 6        |
| 5.2. Buffers . . . . .                                                 | 6        |
| 5.3. Bind Groups . . . . .                                             | 6        |
| 5.4. Shader WGSL . . . . .                                             | 7        |
| <b>6. Parte C — WebGPU + TSL</b>                                       | <b>7</b> |
| 6.1. Three.js WebGPU Renderer . . . . .                                | 7        |
| 6.2. Shading con nodos TSL . . . . .                                   | 7        |
| 6.3. Jerarquía con Three.js . . . . .                                  | 8        |
| <b>7. Comparativa de los tres enfoques</b>                             | <b>8</b> |
| <b>8. Estructura del proyecto</b>                                      | <b>8</b> |

|                        |          |
|------------------------|----------|
| <b>9. Despliegue</b>   | <b>9</b> |
| <b>10.Conclusiones</b> | <b>9</b> |

# 1. Objetivo

El objetivo de esta tarea es implementar las jerarquías 3D articuladas de la Tarea 2 —brazo robótico y sistema solar— usando APIs de gráficos de bajo nivel, **sin ningún framework** que abstraiga el pipeline. Al entender cómo funciona cada etapa del pipeline a este nivel se pueden escribir shaders más eficientes para las tareas posteriores.

El proyecto se divide en tres partes:

- **Parte A (50 pts):** WebGL2 + GLSL — pipeline completo gestionado a mano.
- **Parte B (25 pts):** WebGPU + WGSL — API moderna con render pipeline y bind groups.
- **Parte C (25 pts):** WebGPU + TSL — Three.js WebGPU renderer con shaders en nodos.

La demo desplegada está disponible en:

<https://tareasseminario.vercel.app/>

## 2. Escenas implementadas

En las tres partes se implementan las mismas dos escenas, tomadas directamente de la especificación de la Tarea 2:

### 2.1. Brazo robótico articulado

Cadena jerárquica de **5 eslabones más 2 dedos**:

Base → Hombro → Codo → Muñeca → Dedo 1, Dedo 2

Cada articulación rota en su eje local (no en el origen global) usando un *matrix stack* push/pop. La animación automática usa funciones `Math.sin(t)` con amplitudes y fases distintas por articulación, produciendo un ciclo continuo y suave. Los controles del panel permiten desactivar la animación automática y rotar cada articulación de forma independiente con sliders.

### 2.2. Sistema solar

Jerarquía: Sol → **3 planetas** → **1-2 lunas por planeta**.

| Planeta | Lunas             | Característica          |
|---------|-------------------|-------------------------|
| Tierra  | 1 (Luna)          | inclinación axial 23.4° |
| Marte   | 2 (Fobos, Deimos) | color rojizo            |
| Saturno | 1 (Titán)         | anillo toroidal         |

El Sol se comporta como fuente de luz puntual ubicada en el origen. Cada planeta orbita al sol y rota sobre su propio eje con velocidades distintas. Un slider de velocidad global escala todas las velocidades de órbita simultáneamente.

### 3. Biblioteca compartida

Las tres partes reutilizan los mismos módulos matemáticos ubicados en `src/shared/`:

#### 3.1. `mat4.js` — Álgebra de matrices $4 \times 4$

Implementación propia en `Float32Array` de orden column-major, compatible con WebGL y WGSL. Funciones implementadas:

- `multiply`, `translate`, `rotateX/Y/Z`, `scale`
- `perspective` — proyección con *frustum*
- `lookAt` — cámara orientada hacia un punto
- `invert` — inversión para la normal matrix
- `normalMatrix` — transpuesta de la inversa de la parte  $3 \times 3$  superior-izquierda

#### 3.2. `MatrixStack.js` — Pila de matrices

Implementa el patrón clásico `push/pop` para jerarquías de transformaciones. Cada `push()` guarda una copia del estado actual; `pop()` lo restaura. Incluye métodos de conveniencia `translate`, `rotateX/Y/Z` y `scale` que operan sobre la matriz del tope de la pila.

Este stack es el equivalente manual al `Object3D` de Three.js.

#### 3.3. `primitives.js` — Generadores de geometría

Genera arreglos de vértices indexados (posición + normal) para: `createSphere`, `createCylinder`, `createBox`, `createTorus`.

### 3.4. controls-ui.js — Panel de controles

Genera dinámicamente el HTML del panel lateral según la escena activa:

- **Brazo robot:** toggle auto-animación + 6 sliders (base, hombro, codo, muñeca, dedo 1, dedo 2).
- **Sistema solar:** slider de velocidad global ( $0\times$  a  $4\times$ ).

## 4. Parte A — WebGL2 + GLSL

### 4.1. Pipeline manual

Sin ningún framework, cada frame ejecuta:

1. Se calcula la matriz de modelo con el *matrix stack*.
2. Se deriva la *normal matrix* (transpuesta de la inversa) en CPU.
3. Se suben las matrices como uniforms con `gl.uniformMatrix4fv` y `gl.uniformMatrix3fv`.
4. Se vincula el VAO y se llama a `gl.drawElements`.

### 4.2. VAOs y VBOs

Cada malla tiene su propio *Vertex Array Object* que memoriza:

- VBO con datos entrelazados: posición (vec3) + normal (vec3), 6 floats por vértice.
- EBO con índices Uint16Array para triangulación.
- Descripción de atributos: `a_position` en location 0, `a_normal` en location 1.

### 4.3. Shader GLSL ES 3.00

**Vertex shader:** transforma la posición al clip space y calcula la posición en world space y la normal transformada:

```
out vec3 v_worldPos;
out vec3 v_normal;

void main() {
    vec4 worldPos = u_model * vec4(a_position, 1.0);
    v_worldPos = worldPos.xyz;
    v_normal    = u_normalMatrix * a_normal;
    gl_Position = u_projection * u_view * worldPos;
}
```

## Fragment shader — iluminación Phong difusa + Blinn-Phong especular:

```
void main() {
    vec3 N = normalize(v_normal);
    vec3 L = normalize(u_lightPos - v_worldPos);
    vec3 V = normalize(u_viewPos - v_worldPos);
    vec3 H = normalize(L + V);          // half-vector Blinn-Phong

    vec3 ambient = u_ambientColor * u_lightColor * 0.25;
    float diff = max(dot(N, L), 0.0);
    vec3 diffuse = diff * u_diffuseColor * u_lightColor;
    float spec = pow(max(dot(N, H), 0.0), u_shininess);
    vec3 specular = spec * u_specularColor * u_lightColor;

    fragColor = vec4(ambient + diffuse + specular, 1.0);
}
```

El half-vector  $\mathbf{H} = \text{normalize}(\mathbf{L} + \mathbf{V})$  es la característica central de Blinn-Phong, que reemplaza el cálculo del vector de reflexión  $\mathbf{R}$  de Phong clásico y produce mejores resultados con exponentes altos.

## 5. Parte B — WebGPU + WGSL

### 5.1. Inicialización del dispositivo

1. `navigator.gpu.requestAdapter()` — se intenta con *high-performance*, *low-power* y sin preferencia como fallback.
2. `adapter.requestDevice()` — crea el dispositivo lógico.
3. `canvas.getContext('webgpu')` — se configura con `context.configure()`.

### 5.2. Buffers

- **Vertex buffer:** datos entrelazados posición + normal, `GPUBufferUsage.VERTEX`.
- **Index buffer:** índices `Uint32Array`, `GPUBufferUsage.INDEX`.
- **Uniform buffers:** uno por cada bloque de datos uniformes (cámara, luz, modelo, material), `GPUBufferUsage.UNIFORM | COPY_DST`. Se actualizan con `device.queue.writeBuffer()`.

### 5.3. Bind Groups

Los recursos se organizan en dos grupos de binding:

- **Group 0:** Camera (view, projection, viewPos) + Light (position, color). Se crea una vez por escena.

- **Group 1:** **Model** (model matrix, normal matrix) + **Material** (ambient, diffuse, specular, shininess, emissive). Se actualiza antes de cada draw call.

## 5.4. Shader WGSL

El shader WGSL replica la misma iluminación Blinn-Phong. La normal matrix se almacena como tres columnas `vec4` para respetar el alineamiento de 16 bytes de WGSL:

```
@fragment
fn fs_main(in: VertOut) -> @location(0) vec4<f32> {
    let N = normalize(in.normal);
    let L = normalize(light.position - in.worldPos);
    let V = normalize(camera.viewPos - in.worldPos);
    let H = normalize(L + V);

    let ambient = material.ambient * light.color * 0.25;
    let diffuse = max(dot(N, L), 0.0) * material.diffuse * light.color;
    let spec = pow(max(dot(N, H), 0.0), material.shininess);
    let specular = spec * material.specular * light.color;
    return vec4<f32>(ambient + diffuse + specular, 1.0);
}
```

## 6. Parte C — WebGPU + TSL

### 6.1. Three.js WebGPU Renderer

Se usa `WebGPURenderer` de Three.js (`three/webgpu`) en lugar del renderer WebGL estándar. Requiere llamar `await renderer.init()` antes del primer frame.

### 6.2. Shading con nodos TSL

TSL (*Three Shading Language*) permite escribir el shader como un grafo de nodos en JavaScript. El mismo cálculo Blinn-Phong se expresa así:

```
const N = normalize(normalWorld);
const L = normalize(lightPos.sub(positionWorld));
const V = normalize(cameraPosition.sub(positionWorld));
const H = normalize(L.add(V));

const ambient = mul(vec3(0.15, 0.15, 0.2), lightColor).mul(0.25);
const diffuse = mul(max(dot(N, L), 0), mul(diffuseVec, lightColor));
const specular = mul(pow(max(dot(N, H), 0), shininess),
                    mul(specularVec, lightColor));

mat.colorNode = add(add(ambient, diffuse), specular);
```

TSL traduce este grafo al shader WGSL correspondiente en tiempo de compilación.



### 6.3. Jerarquía con Three.js

Para la Parte C se usan objetos `THREE.Group` para la jerarquía, lo que permite delegar las transformaciones encadenadas al motor, mientras el shader personalizado Blinn-Phong sigue siendo responsabilidad del desarrollador.

## 7. Comparativa de los tres enfoques

| Aspecto              | WebGL2 + GLSL                 | WebGPU + WGSL   | WebGPU + TSL           |
|----------------------|-------------------------------|-----------------|------------------------|
| Matrices             | Manual (CPU)                  | Manual (CPU)    | Three.js               |
| Jerarquía            | MatrixStack                   | MatrixStack     | Group/Object3D         |
| Shader               | GLSL ES 3.00                  | WGSL            | Nodos JS               |
| Uniforms             | <code>uniformMatrix4fv</code> | Uniform buffers | <code>uniform()</code> |
| Vértices             | VAO + VBO                     | GPUBuffer       | BufferGeometry         |
| Curva de aprendizaje | Alta                          | Muy alta        | Media                  |

## 8. Estructura del proyecto

```
Tarea_3/
+-- index.html          (pagina de navegacion)
+-- partA/              (WebGL2)
+-- partB/              (WebGPU)
+-- partC/              (TSL)
+-- src/
  +-- shared/
    | +-- mat4.js        (matrices 4x4 column-major)
    | +-- MatrixStack.js (pila push/pop)
    | +-- primitives.js  (sphere, cylinder, box, torus)
    | +-- controls-ui.js (panel de controles HTML)
  +-- webgl2/
    | +-- shaders.js     (GLSL ES 3.00)
    | +-- program.js     (createProgram, createMesh, setUniforms)
    | +-- RobotArm.js
    | +-- SolarSystem.js
  +-- webgpu/
    | +-- shaders.js     (WGSL)
    | +-- pipeline.js    (initGPU, buffers, pipeline, depth)
    | +-- RobotArm.js
    | +-- SolarSystem.js
  +-- tsl/
    +-- RobotArm.js
    +-- SolarSystem.js
```

## 9. Despliegue

El proyecto se construye con **Vite** (`npm run build`) generando un sitio estático en `dist/`. Está desplegado en **Vercel** con configuración automática detectada por el preset de Vite:

- **Build command:** `npm run build`
- **Output directory:** `dist`
- **Root directory:** `Tarea_3`

URL pública: <https://tareasseminario.vercel.app/>

**Nota sobre compatibilidad:** Las Partes B y C requieren un navegador con soporte WebGPU (Chrome 113+ o Edge 113+). En Linux puede ser necesario iniciar Chrome con `-enable-features=Vulkan`. La Parte A (WebGL2) funciona en cualquier navegador moderno.

## 10. Conclusiones

Implementar el mismo pipeline en tres niveles de abstracción distintos permite entender con precisión qué hace cada capa del stack gráfico:

- **WebGL2** obliga a gestionar cada estado de la GPU manualmente: compilación de shaders, atributos de vértices, uniforms y la normal matrix. Es verbose pero sin magia.
- **WebGPU** introduce conceptos modernos como bind groups y uniform buffers explícitos, lo que impone disciplina de alineamiento de memoria pero resulta más predecible y eficiente.
- **TSL** permite escribir el shader como código JavaScript sin abandonar el control sobre la lógica de iluminación, ofreciendo el mejor equilibrio entre productividad y comprensión del pipeline.

La biblioteca compartida (`mat4.js`, `MatrixStack.js`, `primitives.js`) demuestra que las matemáticas de transformaciones son independientes de la API de renderizado, lo que facilita portar las mismas escenas entre WebGL2 y WebGPU con cambios mínimos en la lógica de alto nivel.